```python
import numpy as np

student_scores = np.array([
    [80, 75, 85, 70],
    [90, 85, 80, 75],
    [70, 80, 90, 85],
    [85, 90, 95, 80]
])

subjects = ["Math", "Science", "English", "History"]

average = np.mean(student_scores, axis=0)

for i in range(4):
    print(subjects[i], "Average =", average[i])

highest = np.argmax(average)
print("Highest average subject:", subjects[highest])
```

```
Math Average = 81.25
Science Average = 82.5
English Average = 87.5
History Average = 77.5
Highest average subject: English
```

```python
student_scores = np.array([
    [80, 75, 85, 70],
    [90, 85, 80, 75],
    [70, 80, 90, 85],
    [85, 90, 95, 80]
])

subjects = ["Math", "Science", "English", "History"]

average = np.mean(student_scores, axis=0)

for i in range(4):
    print(subjects[i], "Average =", average[i])

highest = np.argmax(average)
print("Highest average subject:", subjects[highest])
```

```
Math Average = 81.25
Science Average = 82.5
English Average = 87.5
History Average = 77.5
Highest average subject: English
```

```python
import numpy as np

prices = np.array([
    [100, 120, 150],
    [80, 90, 110],
    [200, 180, 160]
])

average_price = np.mean(prices)

print("Average price =", average_price)
```

```
Average price = 132.22222222222223
```

```python
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

data = pd.DataFrame({
    "Size": [800, 1000, 1200, 1500, 1800, 2000],
    "Price": [160, 200, 240, 300, 360, 400]
})

X = data[["Size"]]
y = data["Price"]

model = LinearRegression()
model.fit(X, y)

predicted = model.predict(X)

print("R-squared =", r2_score(y, predicted))

plt.scatter(data["Size"], data["Price"])
plt.plot(data["Size"], predicted)
plt.xlabel("House Size")
plt.ylabel("Price")
plt.title("House Size vs Price")
plt.show()

new_size = float(input("Enter house size: "))
print("Predicted price =", model.predict([[new_size]])[0])
```

```
R-squared = 1.0
```



```python
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

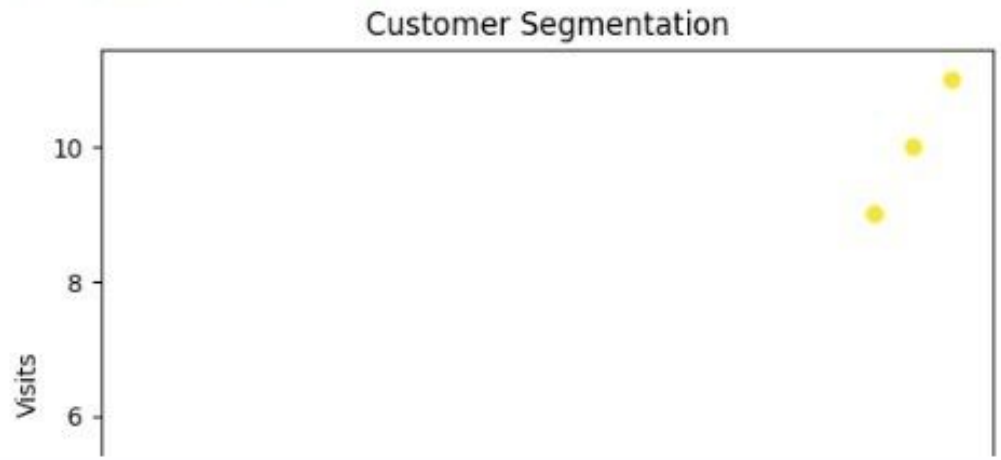
data = pd.DataFrame({
    "Spending": [1000, 1200, 1100, 5000, 5200, 4800],
    "Visits": [2, 3, 2, 10, 11, 9]
})

model = KMeans(n_clusters=2, random_state=1, n_init=10)
data["Cluster"] = model.fit_predict(
    data[["Spending", "Visits"]]
)

print(data)

plt.scatter(
    data["Spending"],
    data["Visits"],
    c=data["Cluster"]
)
plt.xlabel("Spending")
plt.ylabel("Visits")
plt.title("Customer Segmentation")
plt.show()
```

```
   Spending  Visits  Cluster
0      1000       2        0
1      1200       3        0
2      1100       2        0
3      5000      10        1
4      5200      11        1
5      4800       9        1
```



```python
from sklearn.metrics import (
    accuracy_score, precision_score,
    recall_score, f1_score
)

# Actual and predicted values
y_test = [0, 1, 1, 0, 1, 0, 1, 1]
y_pred = [0, 1, 0, 0, 1, 0, 1, 0]

print("Accuracy =", accuracy_score(y_test, y_pred))
print("Precision =", precision_score(y_test, y_pred))
print("Recall =", recall_score(y_test, y_pred))
print("F1-score =", f1_score(y_test, y_pred))
```

```
Accuracy = 0.75
Precision = 1.0
Recall = 0.6
F1-score = 0.75
```

```python
import numpy as np
from sklearn.tree import DecisionTreeRegressor

# Features: mileage, age
X = np.array([
    [20000, 2], [30000, 3], [40000, 4],
    [60000, 5], [80000, 7], [100000, 9]
])

price = np.array([18000, 17000, 15000, 13000, 10000, 7000])

model = DecisionTreeRegressor(max_depth=3, random_state=1)
model.fit(X, price)

mileage = float(input("Mileage: "))
age = float(input("Age: "))

prediction = model.predict([[mileage, age]])

print("Predicted price =", prediction[0])
```

```
Mileage: 170
Age: 19
Predicted price = 7000.0
```

```python
import numpy as np
from sklearn.linear_model import LogisticRegression

# Features: usage minutes, contract months
X = np.array([
    [100, 24], [120, 24], [80, 12],
    [300, 6], [280, 6], [350, 3]
])

# 0 = Not churned, 1 = Churned
y = np.array([0, 0, 0, 1, 1, 1])

model = LogisticRegression()
model.fit(X, y)

usage = float(input("Usage minutes: "))
contract = float(input("Contract duration: "))

prediction = model.predict([[usage, contract]])

if prediction[0] == 1:
    print("Customer may churn")
else:
    print("Customer may not churn")

# Key idea: logistic regression predicts a binary outcome such as churn/not churn.
# Question 28: K-Means Customer Segmentation
import numpy as np
from sklearn.cluster import KMeans

X = np.array([
    [1000, 2],
    [1200, 3],
    [1100, 2],
    [5000, 10],
    [5200, 11],
    [4800, 9]
])

model = KMeans(n_clusters=2, random_state=1, n_init=10)
model.fit(X)

new_customer = [[1500, 3]]
cluster = model.predict(new_customer)
```

```python
import numpy as np
from sklearn.linear_model import LinearRegression

# Area in square feet
X = np.array([[1000], [1200], [1500], [1800], [2000]])
# House prices
y = np.array([200000, 240000, 300000, 360000, 400000])

model = LinearRegression()
model.fit(X, y)

area = float(input("Enter house area: "))

prediction = model.predict([[area]])

print("Predicted house price =", prediction[0])
```

```
Enter house area: 3
Predicted house price = 599.999999999942
```

```
y = [0, 0, 0, 1, 1, 1]

model = KNeighborsClassifier(n_neighbors=3)
model.fit(X, y)

patient = [[7, 8]]

prediction = model.predict(patient)

if prediction[0] == 1:
    print("Patient may have the condition")
else:
    print("Patient may not have the condition")
```

```
Patient may have the condition
```

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier

iris = load_iris()

X = iris.data
y = iris.target

model = DecisionTreeClassifier(random_state=1)
model.fit(X, y)

sepal_length = float(input("Sepal length: "))
sepal_width = float(input("Sepal width: "))
petal_length = float(input("Petal length: "))
petal_width = float(input("Petal width: "))

new_flower = [[
    sepal_length, sepal_width,
    petal_length, petal_width
]]

prediction = model.predict(new_flower)

print("Predicted species:", iris.target_names[prediction[0]])
```

```
Sepal length: 2
Sepal width: 3
Petal length: 3
Petal width: 2
Predicted species: versicolor
```

Terminal

```python
import pandas as pd
from scipy import stats
import os

# Programmatically generate a sample customer_reviews.csv file to resolve FileNotFoundError
if not os.path.exists("customer_reviews.csv"):
    sample_reviews = pd.DataFrame({
        "rating": [5, 4, 5, 3, 4, 5, 2, 4, 5, 4, 5, 3, 5, 4, 5]
    })
    sample_reviews.to_csv("customer_reviews.csv", index=False)

data = pd.read_csv("customer_reviews.csv")
ratings = data["rating"].dropna()

confidence = 0.95
mean = ratings.mean()
sem = stats.sem(ratings)

ci = stats.t.interval(
    confidence, len(ratings) - 1,
    loc=mean, scale=sem
)

print("Mean rating =", mean)
print("95% Confidence interval =", ci)
```

```
Mean rating = 4.2
95% Confidence interval = (np.float64(3.67882292920985), np.float64(4.721177070790151))
```

```python
from scipy.stats import ttest_ind
import matplotlib.pyplot as plt

control = np.array([5, 6, 7, 5, 6, 4, 7])
treatment = np.array([8, 9, 10, 9, 11, 8, 10])

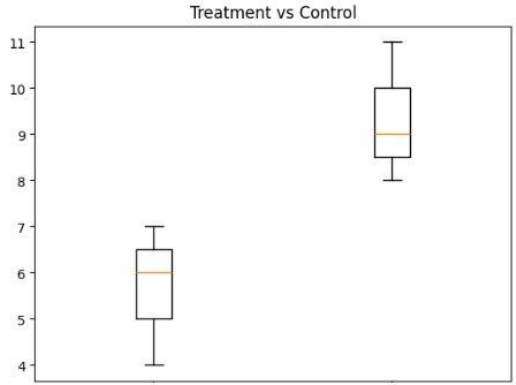
t_stat, p_value = ttest_ind(treatment, control)

print("t-statistic =", t_stat)
print("p-value =", p_value)

if p_value < 0.05:
    print("Significant difference")
else:
    print("No significant difference")

plt.boxplot([control, treatment], labels=["Control", "Treatment"])
plt.title("Treatment vs Control")
plt.show()
```

```
t-statistic = 6.004805767690768
p-value = 6.171074121437904e-05
Significant difference
/tmp/ipykernel_1731/4064839176.py:18: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be dropped in 3.11.
  plt.boxplot([control, treatment], labels=["Control", "Treatment"])
```




Terminal

```python
import pandas as pd
import numpy as np
from scipy import stats
import os

# Create a sample rare_elements.csv file if it does not exist to prevent FileNotFoundError
if not os.path.exists("rare_elements.csv"):
    np.random.seed(42)
    # Simulating some rare element concentration values
    sample_data = pd.DataFrame({"element_concentration": np.random.normal(loc=15.5, scale=3.0, size=100)})
    sample_data.to_csv("rare_elements.csv", index=False)

data = pd.read_csv("rare_elements.csv")
values = data.iloc[:, 0].dropna()

n = int(input("Enter sample size: "))
confidence = float(input("Enter confidence level (for example 0.95): "))
precision = float(input("Enter desired precision: "))

sample = values.sample(n=min(n, len(values)), random_state=1)

mean = sample.mean()
sem = stats.sem(sample)

ci = stats.t.interval(
    confidence, len(sample) - 1, loc=mean, scale=sem
)

print("Point estimate =", mean)
print("Confidence interval =", ci)
print("Desired precision =", precision)
```

```
Enter sample size: 4
Enter confidence level (for example 0.95): 0.97
Enter desired precision: 7
Point estimate = 14.20342711386886
Confidence interval = (np.float64(10.505706976708495), np.float64(17.901147251029226))
Desired precision = 7.0
```

```python
import numpy as np
from scipy import stats

drug = np.array([8, 10, 12, 9, 11, 13, 10, 12])
placebo = np.array([3, 5, 4, 6, 2, 5, 4, 3])

def confidence_interval(data):
    mean = np.mean(data)
    sem = stats.sem(data)
    interval = stats.t.interval(
        0.95, len(data) - 1, loc=mean, scale=sem
    )
    return interval

print("Drug 95% CI:", confidence_interval(drug))
print("Placebo 95% CI:", confidence_interval(placebo))
```

```
Drug 95% CI: (np.float64(9.216289685290807), np.float64(12.033710314709193))
Placebo 95% CI: (np.float64(2.9053916697328868), np.float64(5.094608330267113))
```

```python
import numpy as np
from scipy.stats import ttest_ind

design_A = np.array([0.10, 0.12, 0.11, 0.13, 0.09, 0.12])
design_B = np.array([0.14, 0.15, 0.13, 0.16, 0.12, 0.14])

t_stat, p_value = ttest_ind(design_A, design_B)

print("t-statistic =", t_stat)
print("p-value =", p_value)

if p_value < 0.05:
    print("Statistically significant difference")
else:
    print("No statistically significant difference")
```

```
t-statistic = -3.3999999999999995
p-value = 0.006771072424337956
Statistically significant difference
```

```python
import matplotlib.pyplot as plt
import scipy.stats as stats

data = pd.DataFrame({
    "Age": [20, 22, 25, 28, 30, 32, 35, 40],
    "Fat": [15, 18, 20, 22, 24, 27, 30, 32]
})

print("Mean:")
print(data.mean())

print("Median:")
print(data.median())

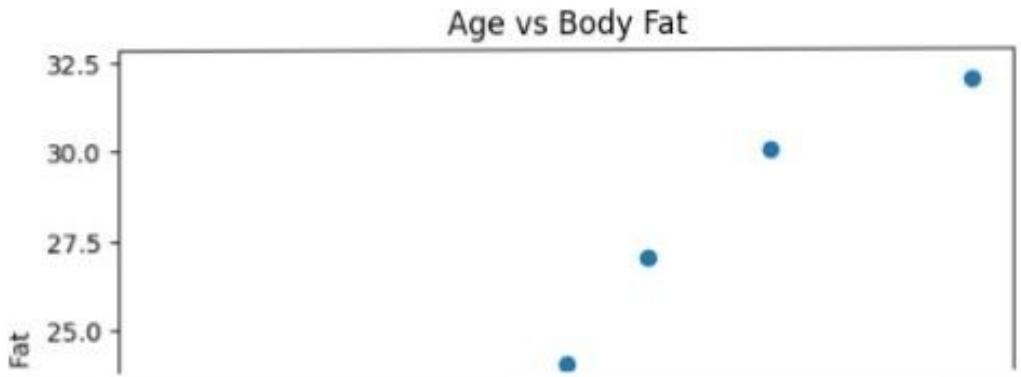
print("Standard deviation:")
print(data.std())

data[["Age", "Fat"]].boxplot()
plt.title("Boxplots")
plt.show()

plt.scatter(data["Age"], data["Fat"])
plt.xlabel("Age")
plt.ylabel("Body Fat")
plt.title("Age vs Body Fat")
plt.show()

stats.probplot(data["Age"], dist="norm", plot=plt)
plt.title("Q-Q Plot of Age")
plt.show()
```



```python
import matplotlib.pyplot as plt

months = [
    "Jan",
    "Feb",
    "Mar",
    "Apr",
    "May",
    "Jun"
]
temperature = [25, 27, 29, 31, 32, 30]
rainfall = [20, 15, 30, 80, 120, 100]

plt.plot(months, temperature, marker="o")
plt.title("Monthly Temperature")
plt.xlabel("Month")
plt.ylabel("Temperature")
plt.show()

plt.scatter(months, rainfall)
plt.title("Monthly Rainfall")
plt.xlabel("Month")
plt.ylabel("Rainfall")
plt.show()

# Question 13: Word Frequency from a Text File
from collections import Counter
import re

# Create a sample text file to prevent FileNotFoundError
with open("sample_text.txt", "w") as file:
  file.write("This is a sample text file. This file is used to demonstrate word frequency counting in Python.")

with open("sample_text.txt", "r") as file:
  text = file.read().lower()

words = re.findall(r"\b\w+\b", text)

frequency = Counter(words)

for word, count in frequency.most_common():
  print(word, ":", count)
```



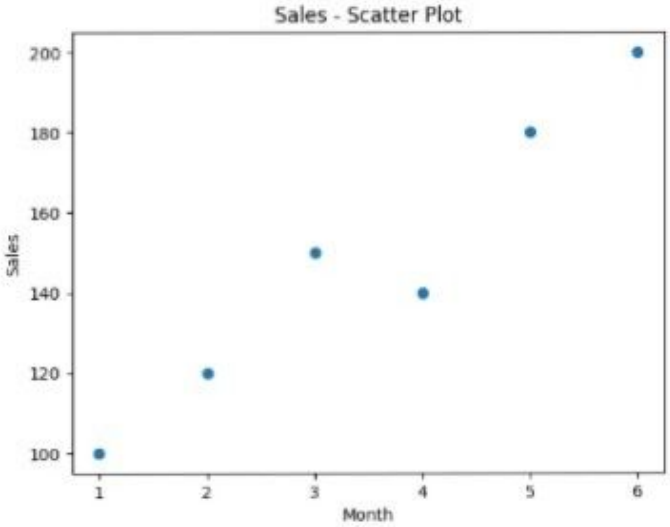

Terminal

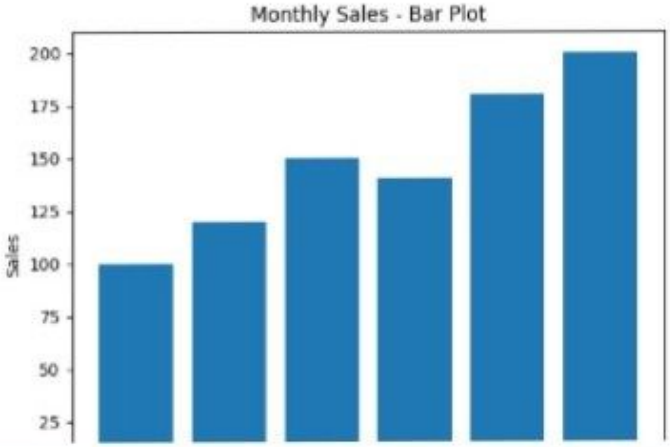
```
plt.title("Sales over Time")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.show()

plt.scatter(months, sales)
plt.title("Sales - Scatter Plot")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.show()

plt.bar(months, sales)
plt.title("Monthly Sales - Bar Plot")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.show()
```



```
for text in data["feedback"].dropna():
    text = text.lower()
    words.extend(re.findall(r"\b[a-z]+\b", text))

words = [word for word in words if word not in stop_words]

frequency = Counter(words)

n = int(input("Enter N: "))
top_words = frequency.most_common(n)

print(top_words)

names = [x[0] for x in top_words]
counts = [x[1] for x in top_words]

plt.bar(names, counts)
plt.xticks(rotation=45)
plt.title("Top Words in Customer Feedback")
plt.show()
```

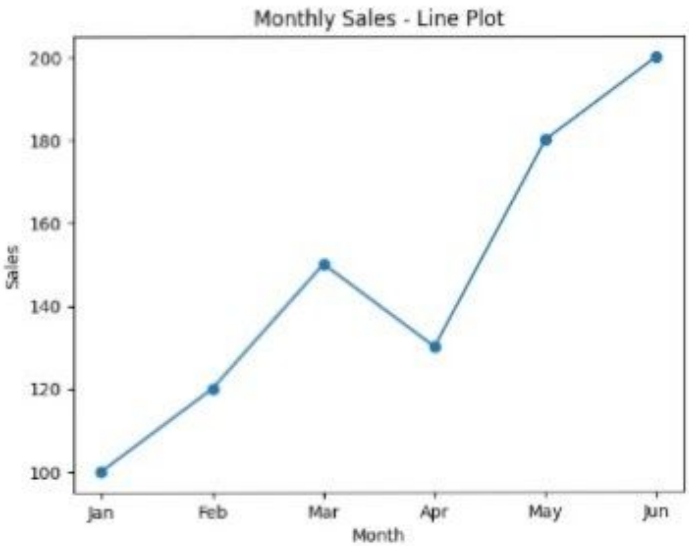
```
Enter N: 8
[('product', 2), ('very', 2), ('good', 2), ('quality', 2), ('easy', 1), ('use', 1), ('excellent', 1), ('customer', 1)]
```

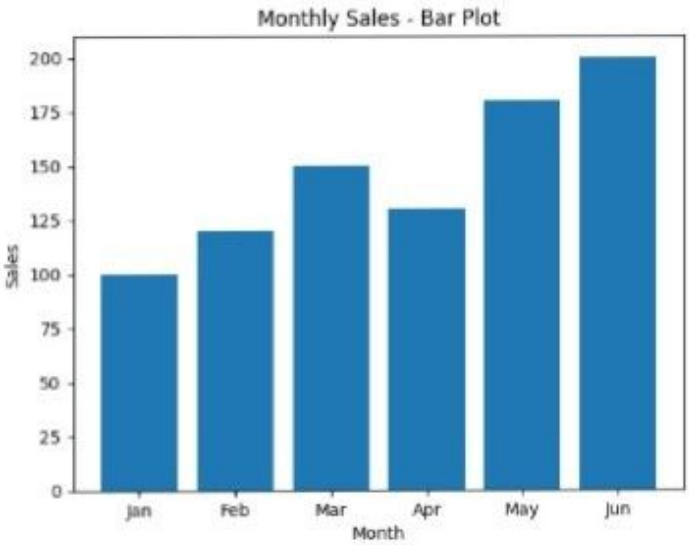


```
months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"]
sales = [100, 120, 150, 130, 180, 200]

plt.plot(months, sales, marker="o")
plt.title("Monthly Sales - Line Plot")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.show()

plt.bar(months, sales)
plt.title("Monthly Sales - Bar Plot")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.show()
```



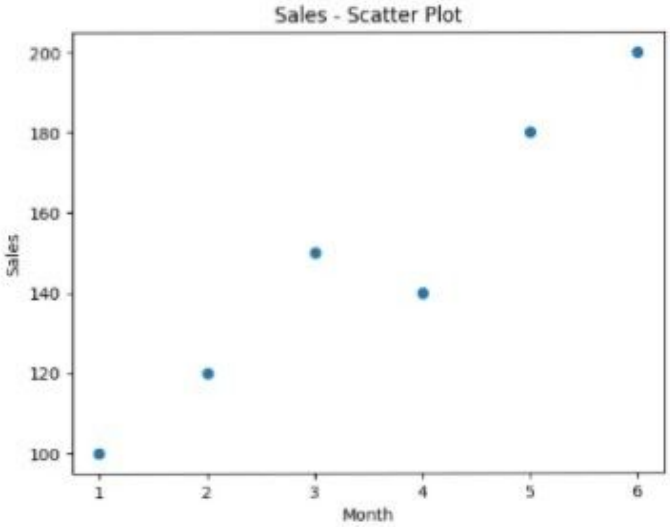



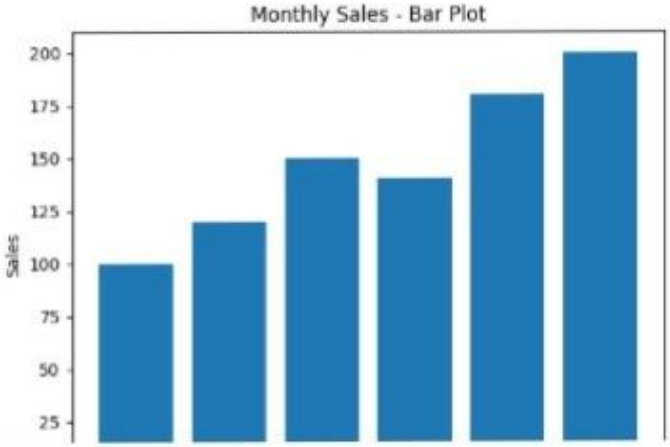
```
plt.title("Sales over Time")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.show()

plt.scatter(months, sales)
plt.title("Sales - Scatter Plot")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.show()

plt.bar(months, sales)
plt.title("Monthly Sales - Bar Plot")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.show()
```



```python
import pandas as pd

property_data = pd.DataFrame({
    "Location": ["Chennai", "Chennai", "Madurai", "Madurai", "Chennai"],
    "Bedrooms": [3, 5, 4, 6, 5],
    "Area": [1200, 1800, 1500, 2200, 2000],
    "Price": [300000, 450000, 350000, 500000, 480000]
})

print("Average price by location:")
print(property_data.groupby("Location")["Price"].mean())

print("Properties with more than 4 bedrooms:")
print((property_data["Bedrooms"] > 4).sum())

largest = property_data.loc[property_data["Area"].idxmax()]
print("Property with largest area:")
print(largest)
```

```
Average price by location:
Location
Chennai    410000.0
Madurai    425000.0
Name: Price, dtype: float64
Properties with more than 4 bedrooms:
3
Property with largest area:
Location    Madurai
Bedrooms          6
Area           2200
Price        500000
Name: 3, dtype: object
```

```python
import pandas as pd
import matplotlib.pyplot as plt

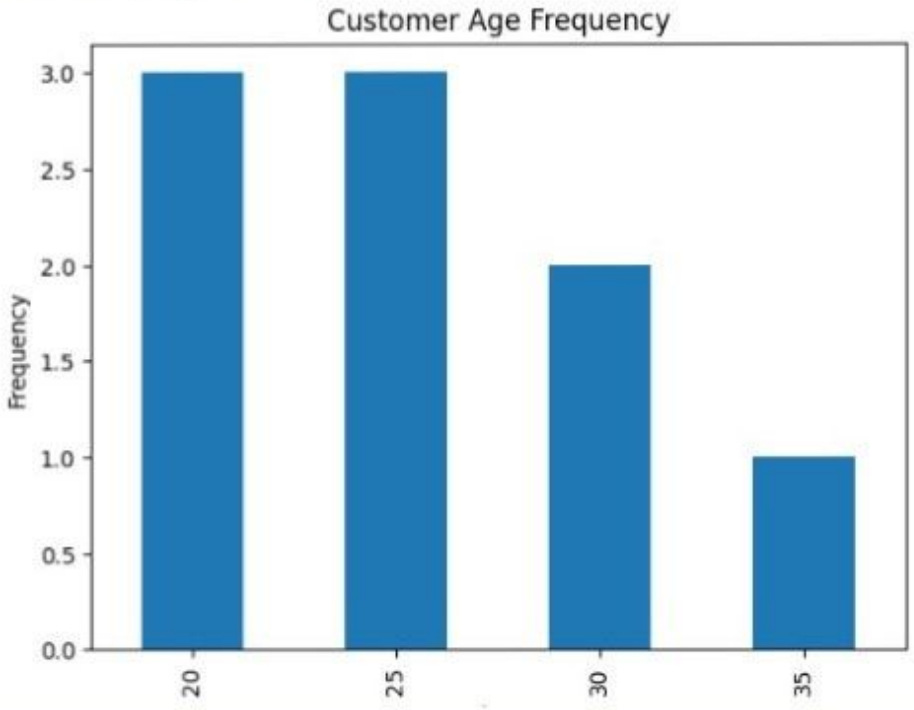
data = pd.DataFrame({
    "Age": [20, 25, 20, 30, 25, 20, 35, 30, 25]
})

frequency = data["Age"].value_counts().sort_index()

print(frequency)

frequency.plot(kind="bar")
plt.title("Customer Age Frequency")
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.show()
```

```
Age
20    3
25    3
30    2
35    1
Name: count, dtype: int64
```

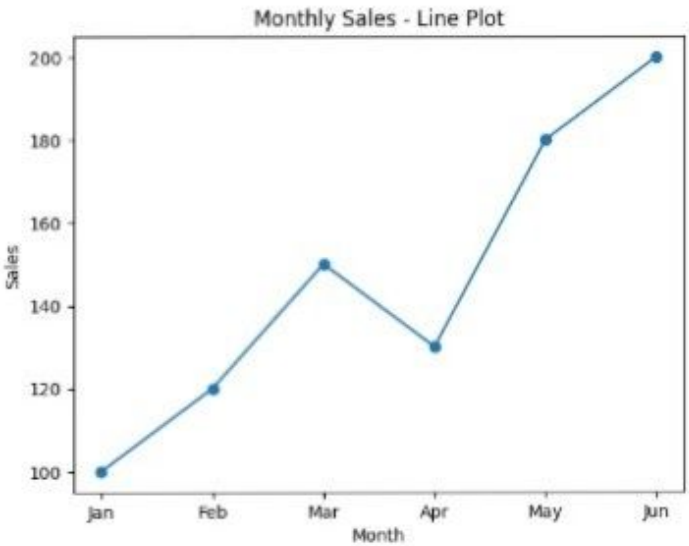


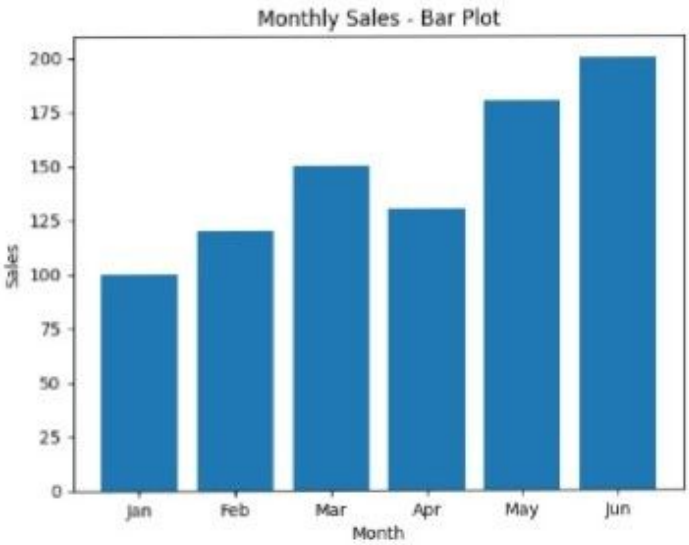

Terminal

```
months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"]
sales = [100, 120, 150, 130, 180, 200]

plt.plot(months, sales, marker="o")
plt.title("Monthly Sales - Line Plot")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.show()

plt.bar(months, sales)
plt.title("Monthly Sales - Bar Plot")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.show()
```



```python
import pandas as pd

property_data = pd.DataFrame({
    "Location": ["Chennai", "Chennai", "Madurai", "Madurai", "Chennai"],
    "Bedrooms": [3, 5, 4, 6, 5],
    "Area": [1200, 1800, 1500, 2200, 2000],
    "Price": [300000, 450000, 350000, 500000, 480000]
})

print("Average price by location:")
print(property_data.groupby("Location")["Price"].mean())

print("Properties with more than 4 bedrooms:")
print((property_data["Bedrooms"] > 4).sum())

largest = property_data.loc[property_data["Area"].idxmax()]
print("Property with largest area:")
print(largest)
```

```
Average price by location:
Location
Chennai    410000.0
Madurai    425000.0
Name: Price, dtype: float64
Properties with more than 4 bedrooms:
3
Property with largest area:
Location    Madurai
Bedrooms          6
Area           2200
Price        500000
Name: 3, dtype: object
```

```python
import pandas as pd

sales = pd.DataFrame({
    "Product": ["Pen", "Book", "Pen", "Bag", "Book", "Pen", "Bag"],
    "Quantity": [10, 8, 15, 5, 12, 20, 9]
})

top5 = sales.groupby("Product")["Quantity"].sum()
top5 = top5.sort_values(ascending=False).head(5)

print(top5)
```

```
Product
Pen     45
Book    20
Bag     14
Name: Quantity, dtype: int64
```

```python
import numpy as np

fuel_efficiency = np.array([25, 30, 28, 35, 32])

average = np.mean(fuel_efficiency)

old_model = fuel_efficiency[0]
new_model = fuel_efficiency[1]

improvement = ((new_model - old_model) / old_model) * 100

print("Average fuel efficiency =", average)
print("Percentage improvement =", improvement, "%")
```

```
Average fuel efficiency = 30.0
Percentage improvement = 20.0 %
```

```python
import pandas as pd

order_data = pd.DataFrame({
    "Customer_ID": [1, 1, 2, 2, 3],
    "Order_Date": ["2026-01-02", "2026-01-05", "2026-01-03",
                   "2026-01-10", "2026-01-08"],
    "Product": ["Pen", "Book", "Pen", "Bag", "Book"],
    "Quantity": [2, 3, 5, 1, 4]
})

print("Orders by customer:")
print(order_data.groupby("Customer_ID").size())

print("\nAverage quantity by product:")
print(order_data.groupby("Product")["Quantity"].mean())

order_data["Order_Date"] = pd.to_datetime(order_data["Order_Date"])
print("\nEarliest date:", order_data["Order_Date"].min())
print("Latest date:", order_data["Order_Date"].max())
```

```
Orders by customer:
Customer_ID
1    2
2    2
3    1
dtype: int64

Average quantity by product:
Product
Bag     1.0
Book    3.5
Pen     3.5
Name: Quantity, dtype: float64

Earliest date: 2026-01-02 00:00:00
Latest date: 2026-01-10 00:00:00
```

```python
import numpy as np

# Columns: Bedrooms, Area, Sale Price
house_data = np.array([
    [3, 1200, 250000],
    [5, 1800, 400000],
    [6, 2200, 500000],
    [4, 1500, 320000],
    [5, 2000, 450000]
])

selected = house_data[house_data[:, 0] > 4]
average_price = np.mean(selected[:, 2])

print("Average sale price =", average_price)
```

```
Average sale price = 450000.0
```

Double-click (or enter) to edit

```python
import numpy as np

sales_data = np.array([10000, 12000, 15000, 18000])

total_sales = np.sum(sales_data)
percentage_increase = ((sales_data[3] - sales_data[0])
                       / sales_data[0]) * 100

print("Total sales =", total_sales)
print("Percentage increase =", percentage_increase, "%")
```

```
Total sales = 55000
Percentage increase = 80.0 %
```